

モジュール利用方法説明資料 (.net版)

作成日 2024年 6月 20日 第 3.7.2 版



変更履歴

版数	作成/更新年月日	変更内容
1.1	2007/09/21	接続モジュールの入れ替えなしで新規決済手段に対応できるようにモジュールを修正したことに伴い、設定ファイルについての説明を修正。
1.2	2007/12/03	クライアント証明書の導入方法について、記載内容を修正。
1.3	2008/02/21	クライアント証明書の導入方法について、記載内容を再修正。
1.4	2009/02/10	新しいクライアント証明書用に記載内容を修正。
1.5	2009/06/05	設定ファイル記載内容等について、記載内容を修正。
1.6	2009/08/27	設定ファイルの配置場所について、記載内容を修正。
1.7	2009/12/24	クライアント証明書インストール方法に関する補足を追記。
1.8	2010/03/08	ファイル伝送機能の説明を追記。
1.9	2010/09/15	windows2008Server+IIS7.5での設定方法を記載
2.0	2010/11/12	設定パラメータについての記載を修正。
2.1	2011/03/29	動作確認済みバージョンを追記
2.2	2011/04/05	証明書のインポート手順を追記
2.3	2011/04/22	設定ファイル項目に「デバッグオプション」を追記。
2.4	2011/04/25	「接続モジュールの利用について 設定ファイル項目」に「PayPal決済」を追記。
2.5	2011/09/29	クライアント証明書に対し、ASP.Net実行ユーザーでのアクセス許可を設定する手順において管理者権限が必要であることを追記。
2.6	2011/11/09	「接続モジュールの利用について 設定ファイル項目」に「電子マネー決済」を追記。
2.7	2012/02/14	証明書のインポート手順、WindowsServer2008の場合のIISの起動設定に追記。 「決済センター接続エラー(E02005) post時における、通信エラー のトラブルシューティング」を追記。
2.8	2012/12/07	エラーコードE02002の発生条件変更。(電文サイズ:10240⇒102400、電文項目数:50⇒無制限)
2.9	2013/06/05	設定ファイルの配置場所について、追記。
3.0	2013/07/17	デバッグオプションが有効な場合に、カード確認番号はマスクが掛かった状態で出力するよう修正。
3.1	2014/07/03	.NET Framework4.5対応
3.2	2014/11/07	デバッグオプションが有効な場合に、カード番号はマスクが掛かった状態で出力するよう修正。
3.3	2017/06/28	2 - 接続モジュールの利用について、「■設定ファイル」にて決済を追加
3.4	2017/10/11	「接続モジュールの利用について 設定ファイル項目」に「Paidy」を追記。
3.5	2018/05/21	クライアント証明書のファイル名を変更。
3.6	2018/06/18	接続先URLに関する記載を廃止。モジュール利用時に接続先URLを意識する必要はありません。
3.7	2018/09/19	タイムアウト値のデフォルト値(35秒)を追記。 サポートver見直しに伴い、WindowsServer2003に関する記述を削除。およびWSEに関する記述を変更。
3.7.1	2024/02/21	「接続モジュールの利用について」の「■クライアント証明書のインポート方法」以降の手順番号を再付番
3.7.2	2024/06/20	■接続モジュール(ペイジェントB2Bモジュール)機能で使用するクラスについて 動作確認済みのバージョンを追加

接続モジュールの利用について

[. Net版]

■ 接続モジュール(バイジェントB2Bモジュール)機能で使用するクラスについて

接続モジュール.Net版は、Visual Basic で実装されています。 ※Visual C#, Visual C++, Visual J# で利用可能確認済
また、以下の環境で動作確認済みです。

OS: Windows Server 2012 R2 Standard、Windows Server 2016 Datacenter
.NET Framework: 4.5、4.8

接続モジュールを利用する場合、以下のクラスを使用します。

jp.co.ks.merchanttool.connectmodule.system.PaygentB2BModule

設定ファイル "modenv.properties" は、環境変数 "CLASSPATH" に設定されているディレクトリ、または「C:/paygent/」の直下
または、IISのアプリケーションディレクトリ(C:/inetpub/wwwroot/サイト/)に配置してください。

設定ファイルは以下の順位で有効になります。優先順位にご注意ください。

1. IISのアプリケーションディレクトリ
2. CLASSPATH
3. C:/paygent/

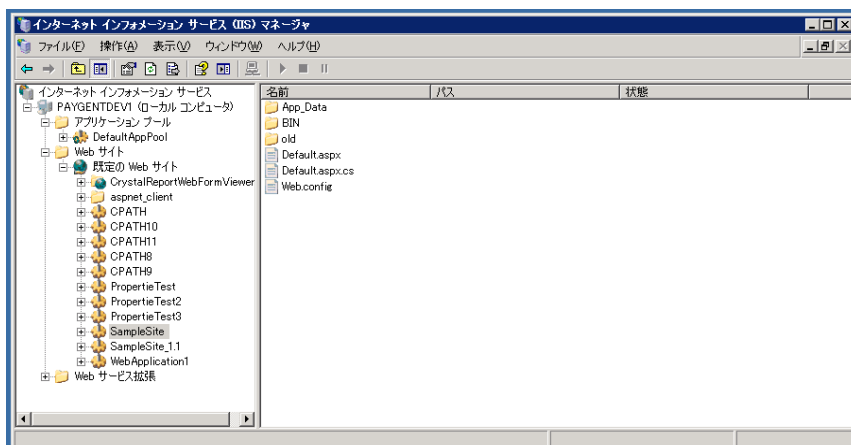
※ 設定ファイルの詳細については、本説明書の「■ 設定ファイル」の項をご確認ください。

IIS上で接続モジュールを使用する場合

必要に応じて接続モジュール単位に設定ファイルを設置することが出来ます。

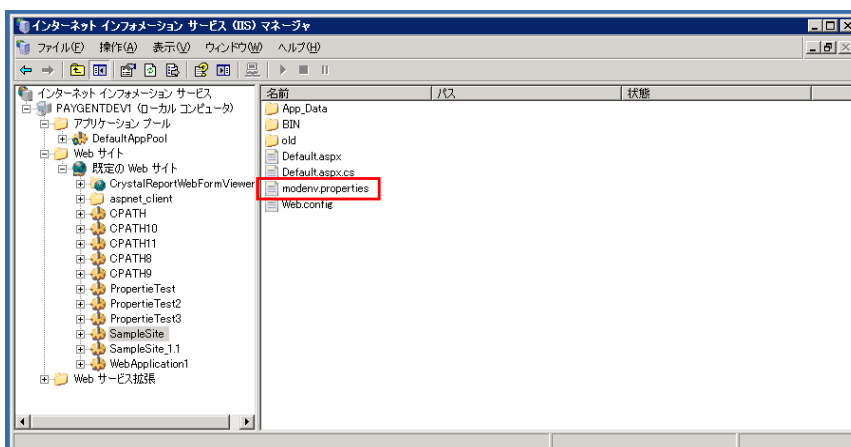
(1) IISマネージャーを開きます

(1) 1. 左パネルのツリーからWEBサイトを選択して下さい



(2) Webサイトにプロパティファイルを設置します

(2) 1. 右クリックメニューからエクスプローラを開き、プロパティファイルを設置します



接続モジュールの利用について

[. Net版]

接続モジュールの設定、移動順は以下の通り

どの電文種別IDにおいても、使用方法は同じ

No	実行内容	パッケージ	クラス	メソッド	備考		
1	オブジェクト生成	jp.co.ks.merchanttool.connectmodule.system	PaygentB2BModule	PaygentB2BModule() Exception: PaygentB2BModuleException	コンストラクタ PaygentB2BModule オブジェクトを取得 No2以降は、このオブジェクトに対して、処理を実施する		
2	モジュール用パラメーターの設定 ※必要な情報を設定			Sub setDefaultId (defaultId As String)	接続ID		
				Sub setDefaultPassword (defaultPassword As String)	接続パスワード		
				Sub setTimeout (timeout As Integer)	セッションタイムアウト値		
				Sub setResultCsv (resultCsv As String)	結果CSVファイル名 ※電文種別ID:027(カード情報照会)、電文種別ID:201(ファイル決済結果照会)の結果をCSVファイルに出力する場合に使用します ※エラーが発生した場合、結果CSVは出力されません		
3	要求電文用引数設定			Sub reqPut (Byval key As String, Byval value As String)	key:POST/パラメータ名 value:設定値 (外部インターフェース仕様書を参照ください)		
4	要求電文用送信ファイルパス設定			Sub setSendFilePath (sendFilePath As String)	送信ファイルのパス ※電文種別ID:200(ファイル決済要求)で使います。 詳細は下記「■ファイル送信について」参照。		
5	要求電文 POST 処理			Sub post() Exception: PaygentB2BModuleException PaygentB2BModuleConnectException	要求電文を、URLに向けて実行		
6	応答電文 処理結果の取得			Function getResultStatus() As String	0: 正常 1: 異常		
7	応答電文 内容を取得 ※結果CSV指定ありの場合を除く			Function hasResNext() As Boolean	応答電文のレコード存在判定 true: 存在する false: 存在しない		
				Function resNext() As IDictionary	応答電文のレコードを取得 Map型 で返される key:POST/パラメータ名 value:設定値		

■ 電文の結果取得種類

結果設定について以下の3種類があります。

① 1レコード単位

Iterator方式で、1レコードをHashtableとして返却する。1件のレスポンスしかない場合でも同様で、操作を統一する。

IDictionary resNext()
boolean hasResNext()

項目はIDictionaryからキーを指定して取得する。

処理結果、レスポンスコード、レスポンス詳細はIDictionaryに含めない。これらの値は、③の方法で取得する。

【注意】 電文種別ID:201(ファイル決済結果照会)はこちらの取得方法を利用できません。

② 複数レコード単位 (CSVファイル出力)

モジュールパラメータの結果CSVファイル名を指定していた場合、通信終了後CSVファイルを出力する。

出力レコードが2レコード以上の場合、①の1レコード単位の取得はできない。

出力レコードが1レコードの場合は、①の1レコード単位の取得は可能とする。

【注意】 電文種別ID:027(カード情報照会)、電文種別ID:201(ファイル決済結果照会)のみこちらの取得方法を利用できます。

③ 処理結果取得

処理結果、レスポンスコード、レスポンス詳細は、それぞれgetterを用意する。

①の1レコード単位で取得した場合も、②のCSVファイル出力した場合も取得できる。

接続モジュールの利用について

[. Net版]

■ファイル送信について

電文種別IDが200(ファイル決済要求)の場合は、外部インターフェース仕様書に記載されている各項目を以下のように設定する。

- ・(共通ヘッダ項目) ... reqPut()により設定。
- ・取引種別 ... reqPut()により設定。
- ・取引ファイルデータ ... ファイルのパスを setSendFilePath に設定。

※接続モジュールをご利用の場合、上記のようにファイルパスを指定してのファイル決済が可能です。
取引ファイルデータには指定したファイルの内容が設定されます。

※お客様の環境によってはサーバの最大POSTデータサイズを変更していただく必要があります。

■接続モジュールの利用手順(サンプル)

【 Visual Basicで利用する場合 】

前提

```
Imports jp.co.ks.merchanttool.connectmodule.system
```

① 接続モジュールのインスタンス取得

```
Dim mdl As PaygentB2BModule = New PaygentB2BModule
```

② ペイジェントへの要求をセット (cf. HashMap) ※電文に必要な要求情報を設定(要求に必要なパラメータを繰り返して設定)

```
mdl.reqPut("telegram_kind", "010")
mdl.reqPut("telegram_version", "1.0")
```

電文種別ID: 200(ファイル決済要求)の場合は、送信ファイルパスをセット

```
mdl.setSendFilePath("")
```

③ ペイジェントへ要求を送信

```
mdl.post()
```

④ 要求結果を取得

```
Dim resultStatus As String = mdl.getResultStatus ' 処理結果 0=正常終了, 1=異常終了
Dim responseCode As String = mdl.getResponseCode ' 異常終了時、レスポンスコードが取得できる
Dim responseDetail As String = mdl.getResponseDetail ' 異常終了時、レスポンス詳細が取得できる
```

- ・ 1件のみの照会結果の場合

```
If mdl.hasResNext() Then
    ' データが存在
    Dim map As IDictionary = mdl.resNext() ' 要求結果取得
    Dim payment_id As String = CStr(map.Item("payment_id")) ' 決済ID取得
    ' 他、応答情報を取得
End If
```

- ・ 複数件の照会結果の場合

```
While mdl.hasResNext()
    ' データが存在する限り、取得
    Dim map As IDictionary = mdl.resNext() ' 要求結果取得
    Dim payment_id As String = CStr(map.Item("payment_id")) ' 決済ID取得
    ' 他、応答情報を取得
End While
```

決済手段(ATM、カード、コンビニ、銀行ネット)を問わず、上記の手順で統一的に利用可能です。

③、④で接続モジュールのExceptionが発生した場合は、発生したExceptionの getCode() メソッドで、エラーコードを取得できます。

【 Visual C# で利用する場合 】

前提

```
using jp.co.ks.merchanttool.connectmodule.system;
```

① 接続モジュールのインスタンス取得

```
PaygentB2BModule module = new PaygentB2BModule();
```

② ペイジェントへの要求をセット (cf. HashMap) ※電文に必要な要求情報を設定 (要求に必要なパラメータを繰り返して設定)

```
module.reqPut("telegram_kind", "010");
module.reqPut("telegram_version", "1.0");
```

電文種別ID:200(ファイル決済要求)の場合は、送信ファイルパスをセット

```
module.setSendFilePath("");
```

③ ペイジェントへ要求を送信

```
module.post();
```

④ 要求結果を取得

```
String resultStatus = module.getResultStatus(); // 処理結果 0=正常終了, 1=異常終了
String responseCode = module.getResponseCode(); // 異常終了時、レスポンスコードが取得できる
String responseDetail = module.getResponseDetail(); // 異常終了時、レスポンス詳細が取得できる
```

- ・ 1件のみの照会結果の場合


```
If (module.hasResNext()) {
        // データが存在
        IDictionary map = module.resNext(); // 要求結果取得
        String payment_id = (String)map["payment_id"]; // 決済ID取得
        // 他、応答情報を取得
      }
```
- ・ 複数件の照会結果の場合


```
while (mdl.hasResNext()) {
        // データが存在する限り、取得
        IDictionary map = module.resNext(); // 要求結果取得
        String payment_id = (String)map["payment_id"]; // 決済ID取得
        // 他、応答情報を取得
      }
```

決済手段 (ATM、カード、コンビニ、銀行ネット) を問わず、上記の手順で統一的に利用可能です。

③、④で接続モジュールのExceptionが発生した場合は、発生したExceptionの `gatCode()` メソッドで、エラーコードを取得できます。

【 Visual C++ で利用する場合 】

前提

```
using namespace jp::co::ks::merchanttool::connectmodule::system;
```

① 接続モジュールのインスタンス取得

```
PaygentB2BModule^ module = gcnew PaygentB2BModule();
```

② ペイジェントへの要求をセット (cf. HashMap) ※電文に必要な要求情報を設定 (要求に必要なパラメータを繰り返して設定)

```
module->reqPut("telegram_kind", "010");
module->reqPut("telegram_version", "1.0");
```

電文種別ID:200(ファイル決済要求)の場合は、送信ファイルパスをセット

```
module->setSendFilePath("");
```

③ ペイジェントへ要求を送信

```
module->post();
```

④ 要求結果を取得

```
String^ resultStatus = module->getResultStatus(); // 処理結果 0=正常終了, 1=異常終了
String^ responseCode = module->getResponseCode(); // 異常終了時、レスポンスコードが取得できる
String^ responseDetail = module->getResponseDetail(); // 異常終了時、レスポンス詳細が取得できる
```

- ・ 1件のみの照会結果の場合


```
if (module->hasResNext()) {
    // データが存在
    IDictionary^ map = module->resNext(); // 要求結果取得
    String^ payment_id = (String^)map["payment_id"]; // 決済ID取得
    // 他、応答情報を取得
}
```
- ・ 複数件の照会結果の場合


```
While (module.hasResNext()) {
    // データが存在する限り、取得
    IDictionary^ map = module->resNext(); // 要求結果取得
    String^ payment_id = (String^)map["payment_id"]; // 決済ID取得
    // 他、応答情報を取得
}
```

決済手段 (ATM、カード、コンビニ、銀行ネット) を問わず、上記の手順で統一的に利用可能です。

③、④で接続モジュールのExceptionが発生した場合は、発生したExceptionの `gatCode()` メソッドで、エラーコードを取得できます。

【 Visual J# で利用する場合 】

前提

```
import jp.co.ks.merchanttool.connectmodule.system.*;
```

① 接続モジュールのインスタンス取得

```
PaygentB2BModule module = new PaygentB2BModule();
```

② ペイジェントへの要求をセット (cf. HashMap) ※電文に必要な要求情報を設定 (要求に必要なパラメータを繰り返して設定)

```
module.reqPut("telegram_kind", "010");
module.reqPut("telegram_version", "1.0");
```

③ ペイジェントへ要求を送信

```
module.post();
```

電文種別ID: 200 (ファイル決済要求) の場合は、送信ファイルパスをセット

```
module.setSendFilePath("");
```

④ 要求結果を取得

```
String resultStatus = module.getResultStatus(); // 処理結果 0=正常終了, 1=異常終了
String responseCode = module.getResponseCode(); // 異常終了時、レスポンスコードが取得できる
String responseDetail = module.getResponseDetail(); // 異常終了時、レスポンス詳細が取得できる
```

・ 1件のみの照会結果の場合

```
If (module.hasResNext()) {
    // データが存在
    IDictionary map = module.resNext(); // 要求結果取得
    String payment_id = (String)map.get_Item("payment_id"); // 決済ID取得
    // 他、応答情報を取得
}
```

・ 複数件の照会結果の場合

```
while (mdl.hasResNext()) {
    // データが存在する限り、取得
    IDictionary map = module.resNext(); // 要求結果取得
    String payment_id = (String)map.get_Item("payment_id"); // 決済ID取得
    // 他、応答情報を取得
}
```

決済手段 (ATM、カード、コンビニ、銀行ネット) を問わず、上記の手順で統一的に利用可能です。

③、④で接続モジュールのExceptionが発生した場合は、発生したExceptionの `gatCode()` メソッドで、エラーコードを取得できます。

接続モジュールの利用について

[. Net版]

■ 接続モジュールエラーコード表

各種エラー	Error Level	Error Code	ErrorName	OutputTiming	ErrorMessage(En)	ErrorMessage(Jp)	
PaygentB2BModuleException (各種エラー用Exception)	設定ファイルなしエラー	ERROR	E01001	設定ファイルが存在しないエラー	post時	Properties file doesn't exist.	設定ファイルが存在しません。
	設定ファイル不正エラー	ERROR	E01002	設定ファイルにおける設定値のエラー	post時	Properties file contains inappropriate value.	設定ファイルにおいて設定値にエラーがあります。
	レスポンスレコードなしエラー	ERROR	E01003	応答電文にレコードが無いエラー	post()後、resNext()にてレコード取得時	Cannot get ResponseData.	応答電文にデータがありません。
				処理結果が取得出来ないエラー	post()後、getResultStatus()にて処理結果取得時	Cannot get ResultStatus.	処理結果がありません。
				レスポンスコードが取得できないエラー	post()後、getResponseCode()にてレスポンスコード取得時	Cannot get ResponseCode.	レスポンスコードがありません。
				レスポンス詳細が取得出来ないエラー	post()後、getResponseDetail()にてレスポンス詳細取得時	Cannot get ResponseDetail.	レスポンス詳細がありません。
	CSV出力エラー	ERROR	E01004	CSV File 出力エラー	post()時、CSV File出力時	CSV file output error.	CSVファイルが出力できません。
	取引ファイルエラー (電文種別ID: 200利用時)	ERROR	E01005	取引ファイル入力エラー	post()時、取引ファイル読込時	Send file I/O error.	取引ファイルの入力処理ができません。
				取引ファイルに指定されたパスにファイルが存在しない	post()時、取引ファイル読込時	Send file not found.	取引ファイルデータCSVファイルパスの設定エラーです。
その他エラー	ERROR	E01901	応答電文の内容解析時等	post()時、応答電文内容解析時等	メッセージは、各言語にて発生するExceptionを出力		
PaygentB2BModuleConnectException (接続エラー用Exception)	モジュールパラメータエラー	ERROR	E02001	接続モジュールに必須なパラメータにエラーがある	post()時	Error indispensable HTTP request value.	必須のモジュールパラメータにエラーがあります。
	電文要求パラメータエラー	ERROR	E02002	要求電文にて、30Byteを超えるキーが設定されている	post()時	HTTP request key must be shorter than 30 bytes.	電文パラメータに30Byteを超えるキーは設定できません。
				要求電文にて、102400Byteを超える値が設定されている	post()時	HTTP request value must be shorter than 102400 bytes.	電文パラメータに102400Byteを超える値は設定できません。
				102400Byteを超える要求電文が送信されている	post()時	The number of HTTP request telegram must be smaller than 102400.	102400byteを超える電文は送信できません。
				要求電文にて、10MByteを超える電文が設定されている(電文種別ID: 200利用時)	post()時	The number of HTTP request telegram must be smaller than 10MB.	電文パラメータに10Mbyteを超える電文は設定できません。
	電文要求パラメータ固定値想定外エラー	ERROR	E02003	要求電文のパラメータに想定外のエラーがある	post()時	HTTP request contains unexpected value.	電文パラメータに想定外の値が設定されています。
証明書エラー	ERROR	E02004	post時における、通信時の証明書確認時に発生	post()時	メッセージは、各言語にて発生するExceptionを出力		

接続モジュールの利用について

[. Net版]

各種エラー		Error Level	Error Code	ErrorName	OutputTiming	ErrorMessage(En)	ErrorMessage(Jp)
PaygentB2BModuleConnectException (接続エラー用 Exception)	決済センター接続エラー	ERROR	E02005	プロキシ経由による接続が出来ない場合	post()時	Cannot connect to PAYGENT via proxy server.[HostName] Proxy server returns [reply Data].	プロキシ・サーバー[HostName] 経由でペイジェントに接続できません。エラーコード:[reply Data]
				post時における、通信エラー ※	post()時	メッセージは、各言語にて発生するExceptionを出力	
	電文処理タイムアウトエラー	ERROR	E02006	post時における、通信タイムアウト時に発生	post()時	メッセージは、各言語にて発生するExceptionを出力	
	応答対応種別エラー	ERROR	E02007	結果CSVファイル名の設定にエラーがある	post()時の、エラーチェックにおいて	CSV file name error.	CSVファイル名の設定エラーです。

※本説明書末尾「決済センター接続エラー (E02005) post時における、通信エラー のトラブルシューティング」もご確認ください

接続モジュールの利用について

[. Net版]

■ 設定ファイル

マーチャント単位で設定されるものと、デフォルト値として登録しておくパラメータを設定ファイルに保存しておきます。
このファイルは、接続モジュール初回起動時のみ読み込みキャッシュし、それ以降はキャッシュを参照します。
そのため、設定ファイルの変更後は、対象のアプリケーションに紐づくアプリケーションプールの再起動が必要となります。

No	項目名	パラメータ名	属性	サイズ byte	必須 項目	項目定義	備考
1	クライアント証明書ファイルパス	.log_output_path .client_file_path	半角英数記号	256	○	クライアント証明書のシリアル番号 ※後述の方法で確認します	
2	Proxyサーバ名	paygentB2Bmodule .proxy_server_name	半角英数字	80	△	IPアドレスか、どちらかを設定	
3	ProxyIPアドレス	paygentB2Bmodule .proxy_server_ip	数値	15	△	サーバ名か、どちらかを設定	
4	Proxyポート番号	paygentB2Bmodule .proxy_server_port	数値	4	△	Proxyを使用する場合は必須	
5	デフォルトID	paygentB2Bmodule .default_id	半角英数字	64	△	モール対応	
6	デフォルトパスワード	paygentB2Bmodule .default_password	半角英数字	64	△	モール対応	
7	タイムアウト値	paygentB2Bmodule .timeout_value	数値	6	○	ミリ秒数	デフォルト値は35秒
8	ログ出力先	paygentB2Bmodule .log_output_path	半角英数記号	256	○	接続モジュールのログ出力先 ファイル名まで指定してください (ファイルパス、ファイル名は任意です)	ファイルは指定され たファイル名で自動 作成されます
9	デバッグオプション	paygentB2Bmodule .debug_flg	数値	1	○	0:エラー時のみ出力 1:リクエスト/レスポンス情報をログ出力 ※カード番号・カード確認番号はマスク が掛かった状態で出力されます	本番運用では0を 推奨します
10	照会MAX件数	paygentB2Bmodule .select_max_cnt	数値	6	○	照会結果の最大取得件数 最大は、2,000件です	

凡例 必須項目 ○:必須 △:条件付必須

※設定ファイルの「paygentB2Bmodule.url.xx」パラメータは変更しないでください。

接続モジュールの利用について

[. Net版]

■ クライアント証明書のインポート方法

※このインストール方法でないと、ルート証明書、中間証明書とのチェーンが切れてしまい、正常に動作しません。

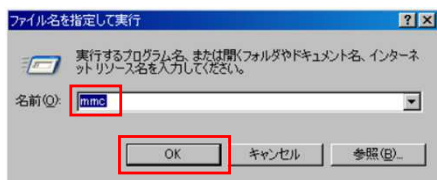
※画面のハードコピー内に表記されている情報(クライアント証明書の名前等)は一部実際のものと異なります。読み替えてご利用ください。

参考URL <http://support.microsoft.com/kb/901183/ja>

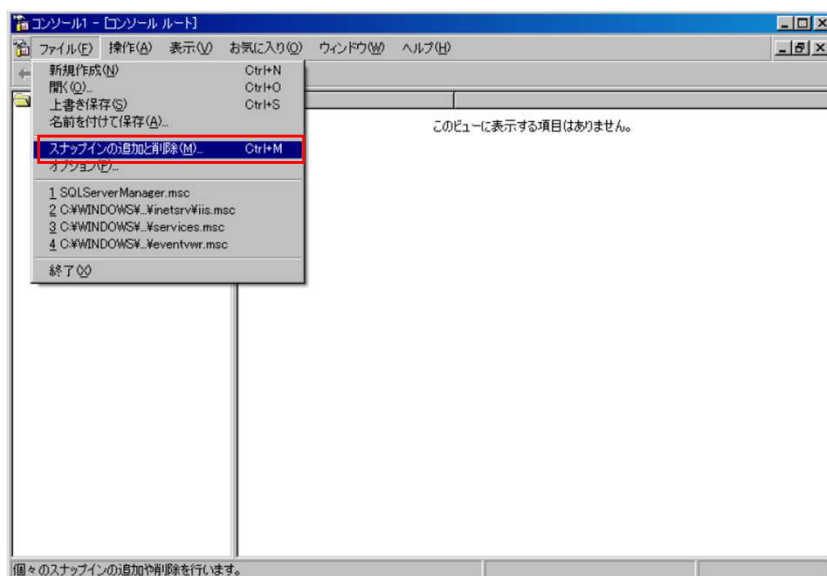
(1) 証明書のインストール、設定を行います

1. 配布された証明書(～.pfx)を、“ローカルコンピュータ”用証明書ストアにインストールします

1.1. [スタート] ボタンをクリックし[ファイル名を指定して実行]、mmc と入力し、[OK] をクリックしてください



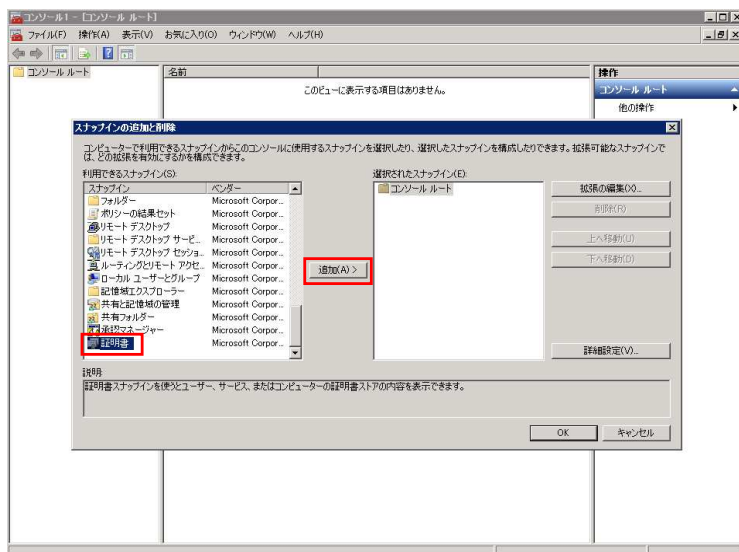
1.2. [ファイル] メニューの [スナップインの追加と削除] をクリックします



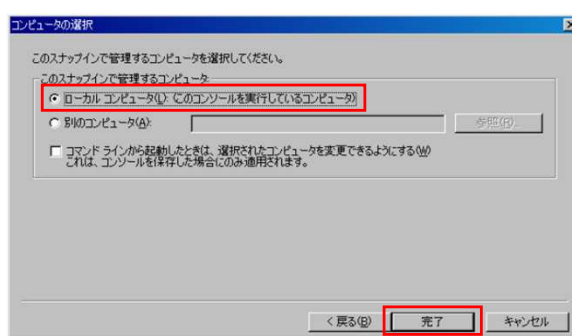
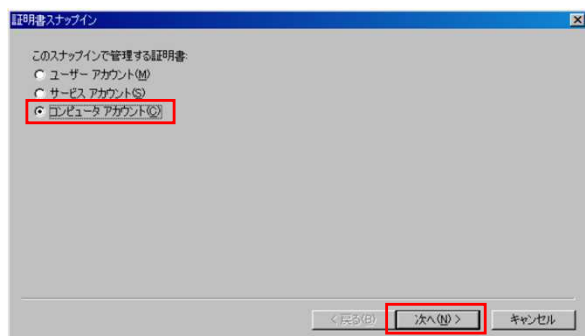
接続モジュールの利用について

[. Net版]

- 1.3. [スナップインの追加と削除] ダイアログ ボックスから[証明書] を選択し、[追加] をクリックします



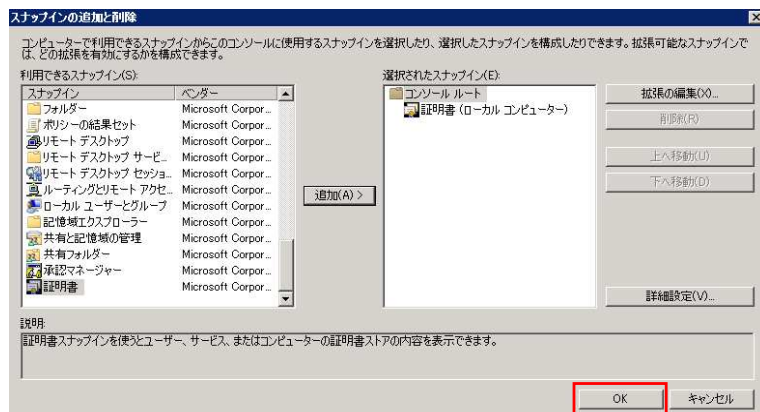
- 1.4. [コンピュータ アカウント] を選択し[次へ]、その後[ローカルコンピュータ]が選択されている状態で[完了]をクリックします



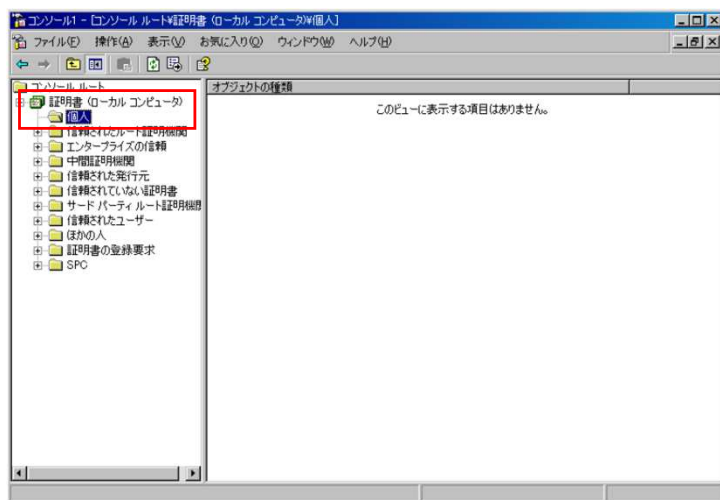
接続モジュールの利用について

[. Net版]

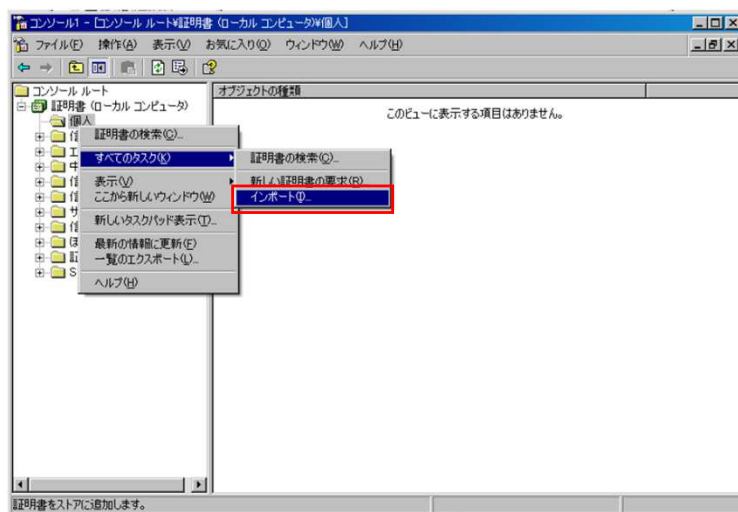
- 1.5. [スナップインの追加と削除] ダイアログボックスで、[OK]をクリックします



- 1.6. クライアント証明書を“ローカルコンピュータ”用証明書ストアにインストールするために、次の手順を実行します
(a) [証明書 (ローカル コンピュータ)] を展開し、[個人] を選択します



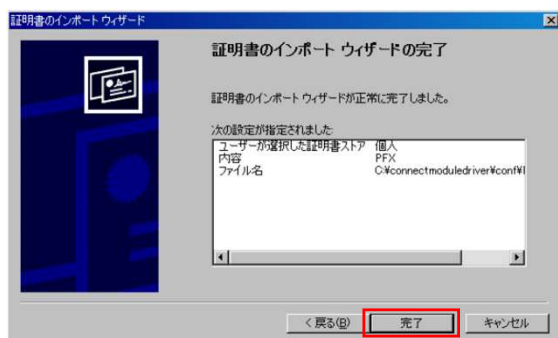
- (b) (a)の[個人]を右クリックし、すべてのタスク - インポート を選択します



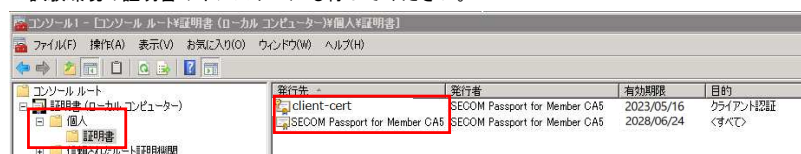
接続モジュールの利用について

[. Net版]

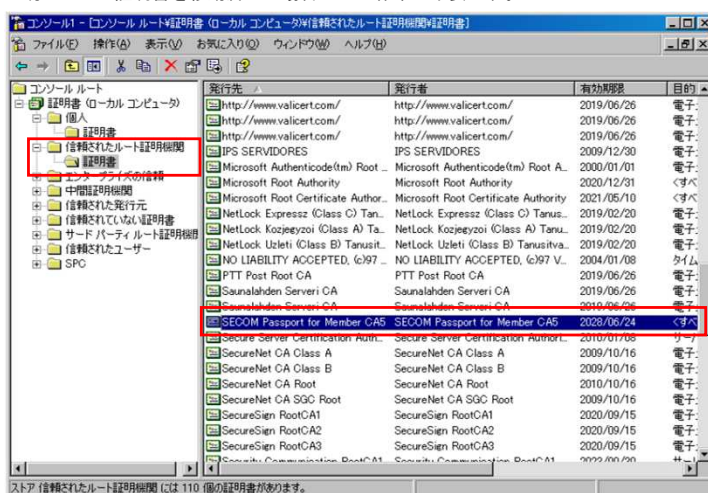
- (c) インポートウィザードが開くので、その中で使用する証明書（～.pfx）を選択しインポートを実施します（途中でパスワードを求められた場合は「changeit」と入力してください）



- (d) [個人]-[証明書]の下に、今回インポートしたファイルができていないことを確認します
 ※ 試験環境の証明書には、クライアント証明書とルート用証明書が含まれているため、2つのファイルが表示されます。
 ※ 本番環境の証明書は、クライアント証明書のみとなります。
 本番環境でサーバーを変更される場合は、試験環境の証明書に含まれるルート証明書が必要になりますので試験環境の証明書のインストールも行ってください。

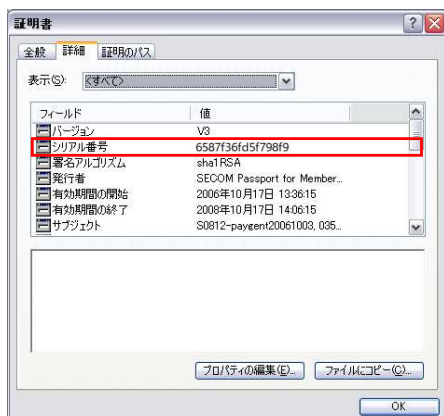


- (e) [個人]-[証明書]にあるルート用証明書 [SECOCM Passport Service Root CA5] を、
 [信頼されたルート証明機関]-[証明書] のディレクトリにドラッグ&ドロップで移動させます
 ※ 既にルート証明書を移動済みの場合はこの作業は不要です。



- (f) 設定ファイル (modenv.properties) の、クライアント証明書ファイルパスに設定するための値として、
 クライアント証明書のシリアル番号を確認します
 [個人]-[証明書]の中に残った証明書をダブルクリックすると[証明書]のウィンドウが開くので詳細タブを選択してください

ここに表示されている「シリアル番号」をメモしておき、
 設定ファイル (modenv.properties) 内に16桁のつながった文字列を転記します（半角スペースは不要）



接続モジュールの利用について

[. Net版]

- 次に[証明書 (ローカル コンピュータ)] の [個人]-[証明書] に残っているクライアント証明書 (接続に利用する証明書) に対し、ASP.NET実行ユーザーでのアクセス許可を設定します

- 2.1. Microsoft Windows HTTP Services Certificate Configuration Tool をダウンロードし、インストールしてください。

このツールを入手するには、次のマイクロソフト Web サイトにアクセスします

<http://www.microsoft.com/downloads/details.aspx?familyid=c42e27ac-3409-40e9-8667-c748e422833f>

インストールディレクトリ (通常: C:\Program Files\Windows Resource Kits\Tools)

- 2.2. インストールディレクトリに移動して、コマンドプロンプトで以下のコマンドを実行します
(ログオンしているユーザが管理者権限を有していない場合、「管理者として実行」でコマンドプロンプトを起動してください)

WinHttpCertCfg.exe -g -c LOCAL_MACHINE\MY -s "IssuedToName" -a "ASPNET"

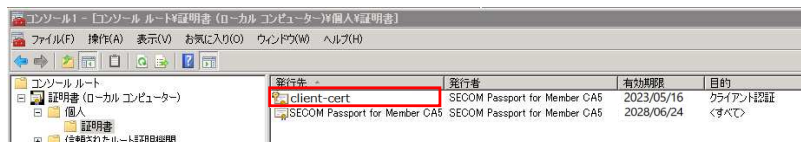
```
C:\Program Files\Windows Resource Kits\Tools>WinHttpCertCfg.exe -g -c LOCAL_MACHINE\MY -s "Module_MdevConnect" -a "ASPNET"
Microsoft (R) WinHTTP Certificate Configuration Tool
Copyright (C) Microsoft Corporation 2001.

Matching certificate:
OID.2.5.4.5=20017061-Mdev002 + CN=Module_MdevConnect
OU="PAYGENT Co., Ltd. PAYGENT B2B Module"
O="PAYGENT Co., Ltd."
C=JP

Granting private key access for account:
```

※「IssuedToName」は、インストールした証明書の発行先を指します

[個人]-[証明書]を表示した際、「発行先」という表示項目があり、この値が「IssuedToName」(発行先)となります



※「ASPNET」は、ASP.NET の実行ユーザーを指します

(IIS5の場合ASP.NETの実行ユーザーはASPNET)

(IIS6の場合ASP.NETの実行ユーザーはNETWORK SERVICE)

WindowsServer 2008 で実施をする場合「.NetFrameworkバージョン 2.0」かつプロセスを「NetworkService」で動作させますので、ASP.NETの実行ユーザーはNETWORK SERVICEを指定します。

- 2.3. コマンドの実行でエラーメッセージが出力されていないこと、
また[個人]-[証明書]を表示しクライアント証明書で右クリック→「すべてのタスク」→「秘密キーの管理」で
ASP.NETの実行ユーザーに秘密キーの読み取り権限が付与されていることを確認します



- (2) WSE(Web Services Enhancements 2.0 for Microsoft .NET) のインストールを行います

※[.NET Framework 3.5] をご利用の場合のみこの手順が必要です。

1. WSE は、.NET Framework の一部としては含まれていない為、次のマイクロソフト Web サイトにアクセスし入手します
<http://www.microsoft.com/downloads/details.aspx?familyid=8070E1DE-22E1-4C78-AB9F-07A7FCF1B6AA&displaylang=en>
2. 入手した Microsoft WSE 2.0 SP3 Runtime.msi をダブルクリックし、インストールします

接続モジュールの利用について【以下はWindowsServer2008、2012の場合に実施】

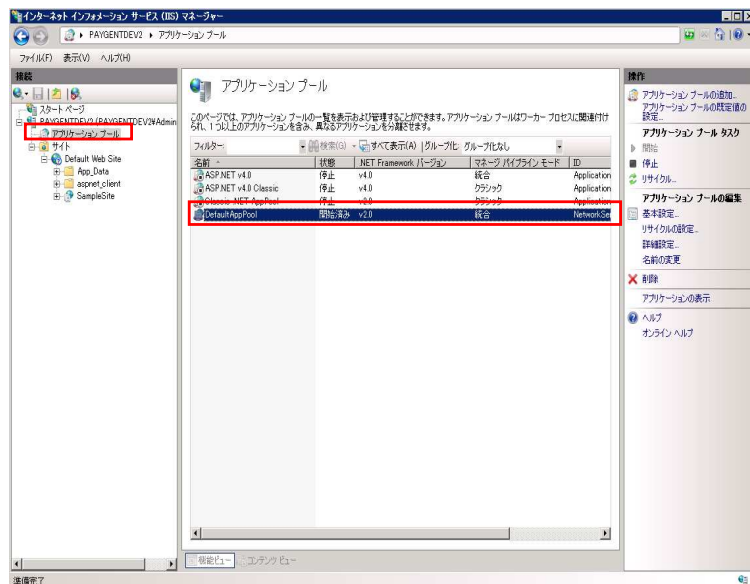
[. Net版]

■ IISの起動設定

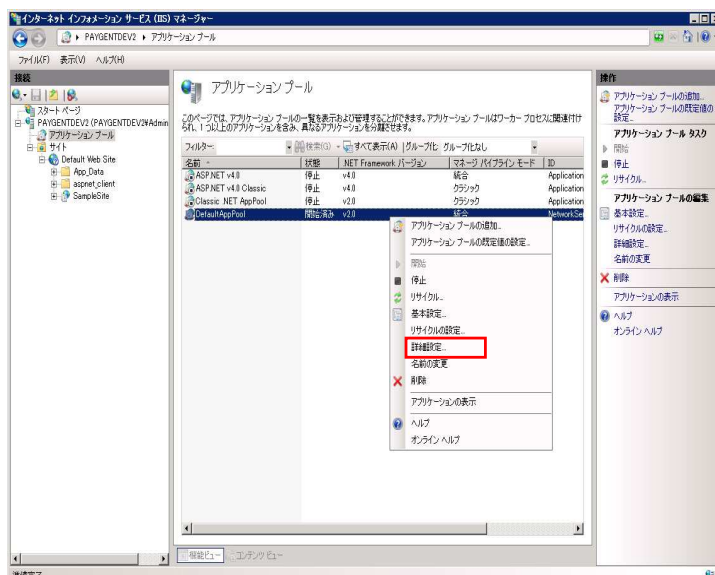
- (1) WindowsServer 2008 で動作させる場合にはアプリケーションプールを.Net2.0系で動作させるための設定が必要となるため、下記を実施します

※以下、ページのサービスを利用するアプリケーションが「DefaultAppPool」配下にあると仮定します。別のアプリケーションプール配下で起動させる場合、そのアプリケーションプール名に読み替えてください。

1. IISマネージャーを開きアプリケーションプールから「DefaultAppPool」を選択します



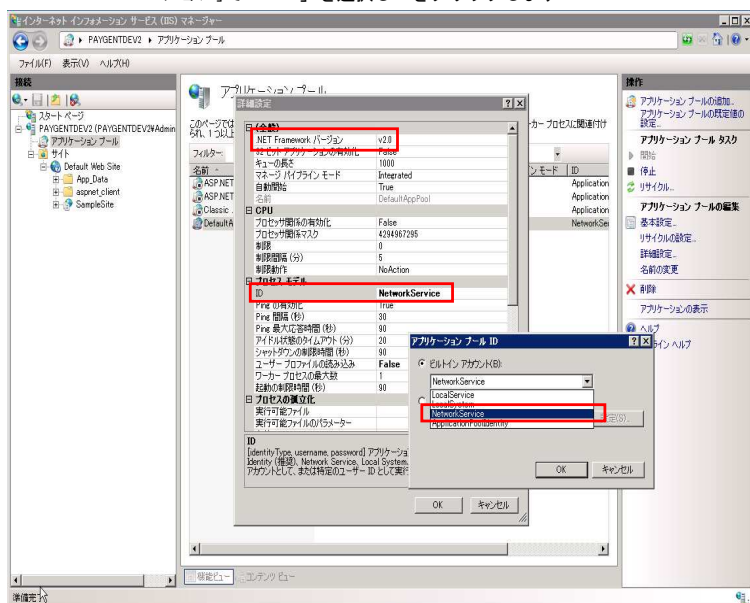
2. 右クリックメニューから「詳細設定」を選びます



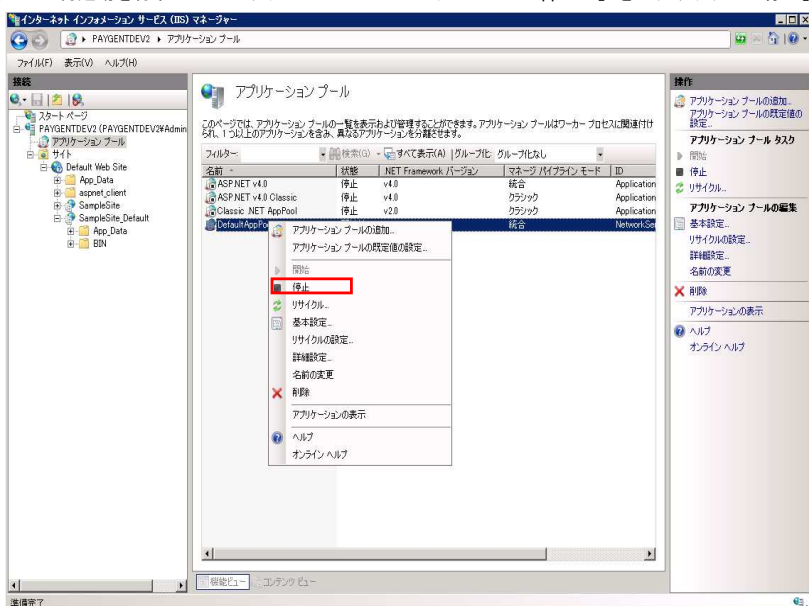
接続モジュールの利用について【以下はWindows Server 2008, 2012の場合に実施】

[. Net版]

3. 「詳細設定」ウィンドウで、[プロセスモデル]の[ID]で「NetWorkService」を選択、
[. NET Framework バージョン]で「v2.0」を選択しOKをクリックします



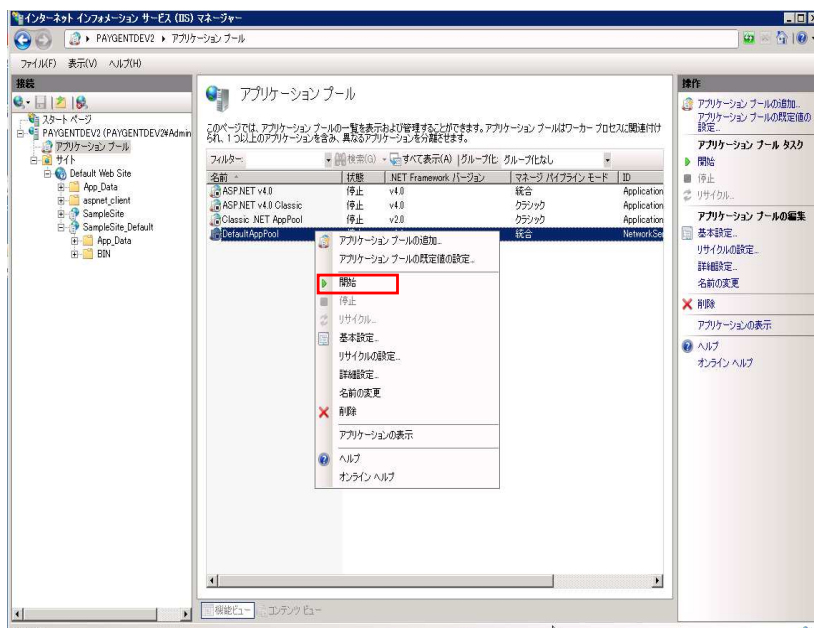
4. 次に、IISの再起動を行うためアプリケーションプールから「DefaultAppPool」を右クリックし「停止」を選択します



接続モジュールの利用について【以下はWindows Server 2008, 2012の場合に実施】

[. Net版]

5. IISを起動させるため、「DefaultAppPool」を右クリックし「開始」を選択します



■ 決済センター接続エラー (E02005) post時における、通信エラー のトラブルシューティング

E02005のエラー発生時、
「保護されているメモリに読み取りまたは書き込み操作を行おうとしました。他のメモリが壊れていることが考えられます。」
といったメッセージが出力されるケースがあります。
上記はASP.NET実行ユーザーが証明書の読み込みに失敗した場合に発生する傾向が強いものです。
以下のチェックポイントを確認する必要があります。

A ASP.NETの実行ユーザーに証明書の秘密キーの読み取り権限が付与されていること

(1) 2.3.を参照してください。

B (WindowsServer2008,2012の場合)

アプリケーションプールのプロセスIDにASP.NETの実行ユーザー (Aと同じユーザー) が指定されていること

(3) 3.を参照してください。

C アプリケーションのWeb設定ファイル(Web.config)の偽装 (Identity impersonation) 設定が行われていないこと

ASP.NETのアプリケーションはWeb.configファイルによってその詳細な動作を制御することが可能です。

Web.configファイル中で下記のように<identity impersonate="true" />の記述があると、
アプリケーションはASP.NETの実行ユーザーではなく特定のログオンユーザーでファイル等にアクセスを試みるようになります。

```
<configuration>
  <system.web>
    <identity impersonate="true" />
  </system.web>
</configuration>
```

本ドキュメントで紹介した手順で導入された証明書は、通常ではASP.NETの実行ユーザーでしか読み込むことは出来ません。
<identity impersonate="false" />とするか、記述自体をコメントアウトもしくは削除してください。

D 設定変更後、アプリケーションプールの再起動が行われていること

(3) 4.および(3) 5.を参照してください (WindowsServer2008,2012における手順ですが、他環境でもほぼ同様です)。